

Velocity Spectrum Module Specification

VS1 and VS2: VACF Transform and Direct Welch Estimation

mdstats

2026-07-22

Contents

Purpose and implementation stage	1
Terminology	2
Provenance	2
Borrowed theory	2
mdstats design	2
Dependency on VACFResult	2
Public result	3
Result identities	3
Public function	4
Normalization contract	4
Reported and biased correlation estimators	4
Transform convention	5
Lag-window contract	5
Zero padding	5
Negative-value policy	5
Algorithm	6
Input constraints and failures	6
Test specification	6
Reuse and non-reuse	7
Direct Welch estimator (VS2)	7
Public function	7
Motive	7
Borrowed algorithm	7
mdstats contribution and attribution boundary	7
Segment and overlap contract	8
Window and detrend contract	8
Weighted self-only periodograms	8
FFT length and frequency grid	9
Atom blocking	9
Algorithm	9
Metadata contract	9
Input constraints and failures	10
VS2 tests	10
Deferred work	10
H0 signature propagation and deep immutability	11
References	11

Purpose and implementation stage

This document specifies

`mdstats/analysis/velocity_spectrum.py`

for release stages VS1 and VS2. The implemented public functions are

```
compute_vacf_spectrum(vacf, ...)
compute_velocity_spectrum(collection, ...)
```

The first transforms an existing VACFResult. The second applies Welch segment averaging directly to uniformly sampled trajectory velocities. Both return the same VelocitySpectrumResult and use the same one-sided density, frequency-axis, weight, tensor, and per-atom conventions.

Terminology

The primary output is a **velocity spectrum** or **velocity spectral density**. It is not automatically called a phonon density of states.

A later explicit normalization layer may produce a finite-temperature VDOS. The label “phonon DOS” is appropriate only for a crystalline or nearly harmonic solid where that interpretation is physically justified.

The output is not an infrared or Raman spectrum. Infrared intensity requires a dipole or charge-current correlation; Raman intensity requires a polarizability correlation.

Provenance

Borrowed theory

The defining relation is the Wiener-Khinchin theorem: the spectral density of a stationary process is the Fourier transform of its autocorrelation [1, 2]. Velocity-correlation spectra have a long history in molecular dynamics, including Rahman’s analysis of liquid argon [3].

Window leakage/resolution trade-offs follow Harris [4]. Direct segment averaging follows Welch [5]. FFT, window, and physical-constant implementations are supplied by SciPy [6].

mdstats design

The following are not attributed to those sources:

- using VACFResult as the only VS1 input;
- explicit reported versus biased finite-origin weighting;
- positive-lag tensor reconstruction through $C_{\alpha\beta}(-t) = C_{\beta\alpha}(t)$;
- one-sided density and THz metadata;
- raw versus per-weight normalization;
- preservation of diagnostic negative lobes by default for VACF transforms;
- atom-blocked weighted self-only Welch accumulation;
- separation of physical drift removal from segmentwise detrending;
- the result schema, validation identities, and provenance payload.

Dependency on VACFResult

The input contains:

```
class VACFResult:
    lag_steps: NDArray[np.int64]          # (L,)
    lag_times: NDArray[np.float64]        # (L,), ps

    scalar_sum: NDArray[np.float64]       # (L,)
    components_sum: NDArray[np.float64]   # (L, 3)
    tensor_sum: NDArray[np.float64] | None # (L, 3, 3)

    per_atom_scalar: NDArray[np.float64] | None
    per_atom_components: NDArray[np.float64] | None
    per_atom_indices: NDArray[np.int64] | None

    n_origins: NDArray[np.int64]          # (L,)
    atom_indices: NDArray[np.int64]
```

```
atom_weights: NDArray[np.float64]
weight_sum: float
```

The spectrum stage requires at least two lags and the exact contiguous lag sequence

```
[0, 1, ..., L - 1]
```

on a uniform physical time grid. A VACF calculated with `lag_stride > 1` must be rejected by VS1; irregular or decimated spectral analysis is a separate problem.

Public result

```
@dataclass(frozen=True, slots=True)
class VelocitySpectrumResult:
    frequencies_thz: NDArray[np.float64]
    angular_frequencies_ps_inv: NDArray[np.float64]
    wavenumbers_cm_inv: NDArray[np.float64]
    energies_mev: NDArray[np.float64]

    scalar_spectrum: NDArray[np.float64]
    component_spectra: NDArray[np.float64]
    tensor_spectrum: NDArray[np.complex128] | None

    per_atom_scalar: NDArray[np.float64] | None
    per_atom_components: NDArray[np.float64] | None
    per_atom_indices: NDArray[np.int64] | None

    atom_indices: NDArray[np.int64]
    atom_weights: NDArray[np.float64]
    weight_sum: float

    estimator: str
    weighting: str
    normalization: str
    correlation_weighting: str | None
    spectral_sidedness: str
    spectral_scaling: str
    spectrum_units: str
    sample_spacing_ps: float
    n_samples: int
    n_fft: int
    window: str | None
    detrend: str | None
    metadata: Mapping[str, Any]
    signature: DynamicsInputSignature | None
```

Result identities

The constructor validates:

$$P_{\text{scalar}}(f) = P_x(f) + P_y(f) + P_z(f).$$

When the tensor exists,

$$\text{diag } \mathbf{P}(f) = (P_x, P_y, P_z),$$

and

$$\mathbf{P}(f) = \mathbf{P}(f)^\dagger.$$

When per-atom results exist,

$$P_i(f) = P_{ix}(f) + P_{iy}(f) + P_{iz}(f).$$

The canonical frequency grid is

$$f_m = \frac{m}{N_{\text{FFT}} \Delta t}.$$

Its first value is zero and its length is

$$\left\lfloor \frac{N_{\text{FFT}}}{2} \right\rfloor + 1.$$

Public function

```
def compute_vacf_spectrum(
    vacf: VACFResult,
    *,
    normalization: Literal["raw", "per_weight"] = "per_weight",
    correlation_weighting: Literal["reported", "biased"] = "reported",
    window: LagWindowInput | None = None,
    zero_pad_to: int | None = None,
    sidedness: Literal["one_sided"] = "one_sided",
    negative_policy: Literal[
        "preserve",
        "clip_roundoff",
        "error",
    ] = "preserve",
    negative_tolerance: float = 1.0e-12,
) -> VelocitySpectrumResult:
    ...
```

Normalization contract

VACFResult stores raw weighted correlation sums.

For normalization="raw", those arrays are transformed without division.

For normalization="per_weight", all scalar, component, tensor, and per-atom correlations are divided by

$$W = \sum_i w_i$$

before transformation.

For a mass-weighted VACF, raw spectral units are

$$\text{amu } \text{\AA}^2/\text{ps},$$

while per-weight units are

$$\text{\AA}^2/\text{ps}.$$

Reported and biased correlation estimators

The stored all-origin estimate is

$$C_{\text{reported}}(k) = \frac{1}{N_{\text{orig}}(k)} \sum_n v_n v_{n+k}.$$

The optional finite-record biased form is

$$C_{\text{biased}}(k) = \frac{N_{\text{orig}}(k)}{N_{\text{orig}}(0)} C_{\text{reported}}(k).$$

For a complete all-origin record this is the familiar triangular lag factor $(T - k)/T$ associated with the finite-record periodogram. The function never applies the factor silently.

The order of operations is:

1. select raw or per-weight normalization;
2. apply reported or biased origin weighting;
3. apply the optional lag taper;
4. reconstruct and transform.

Transform convention

The private `_spectral.py` helper reconstructs negative lags. For a tensor,

$$C_{\alpha\beta}(-t) = C_{\beta\alpha}(t).$$

It then computes

$$S_{\alpha\beta}(f_m) \approx \Delta t \text{RFFT}[g_{\alpha\beta}]_m$$

and applies one-sided density scaling.

The stored canonical frequency is cycles/ps, numerically THz. Angular frequency, cm^{-1} , and meV are derived coordinates for the same bins.

Lag-window contract

Default:

`window=None`

This is rectangular truncation. Built-in alternatives are:

`window="half_hann"`

`window=("half_tukey", alpha)`

Custom arrays are allowed if finite, length-compatible, and equal to one at lag zero. The complete resolved window metadata is stored in the result.

Windowing reduces terminal discontinuity and spectral leakage but broadens peaks. It changes the estimator and is never a plotting-only operation.

Zero padding

`zero_pad_to` is a lower bound on FFT work-array length. The final length is resolved through `scipy.fft.next_fast_len`.

Zero padding:

- refines the frequency grid;
- can improve visual peak interpolation;
- does not add physical information;
- does not improve the resolution set by the useful correlation duration.

Negative-value policy

A transformed finite reported VACF can have negative lobes because of noisy long lags, truncation, or tapering. Such values are not automatically treated as floating-point errors.

preserve Return all real diagonal values unchanged.

clip_roundoff Set only negative values within the configured roundoff threshold to zero; retain material negative values.

error Reject material negative values and clip only roundoff-level negatives.

The threshold is

$$\epsilon_{\text{neg}} = \text{negative_tolerance} \max(1, \max|P|).$$

Only scalar-like diagonal arrays are subject to this policy. Tensor off-diagonal cross spectra remain complex and are never clipped.

Algorithm

1. Validate the input type and contiguous uniform lag grid.
2. Validate estimator, normalization, sidedness, window, and negative policy.
3. Resolve the lag window and FFT length.
4. Build the combined origin/window/normalization lag factor.
5. Transform Cartesian component correlations.
6. Independently transform the scalar correlation and verify the trace.
7. Transform the tensor with transposed negative lags and verify Hermiticity.
8. Transform optional per-atom component and scalar correlations and verify their trace identity.
9. Apply the negative policy to component arrays.
10. Derive scalar arrays from the accepted component arrays.
11. Synchronize the tensor diagonal with the accepted components.
12. Construct THz, rad/ps, cm^{-1} , and meV axes.
13. Return `VelocitySpectrumResult` with source VACF provenance.

Input constraints and failures

The function rejects:

- non-VACFResult input;
- fewer than two lags;
- noncontiguous `lag_steps`;
- a lag-time grid not starting at zero;
- nonuniform or nonincreasing lag times;
- unsupported normalization or weighting modes;
- malformed windows;
- invalid padding;
- unsupported sidedness;
- negative or nonfinite negative tolerance;
- material negative values under `negative_policy="error"`.

The function raises an internal runtime error if independently transformed scalar/component or tensor/component identities disagree. Such an error indicates a programming defect, not a recoverable user condition.

Test specification

`tests/test_velocity_spectrum.py` and `tests/test_spectral.py` provide:

- on-grid cosine peak and exact frequency location;
- direct DFT oracle agreement;
- raw/per-weight ratio checks;
- exact biased-origin weighting checks;
- tensor Hermiticity from nonsymmetric positive-lag cross correlations;
- per-atom/total reconstruction;
- one-sided bin-area identity;
- half-Hann zero-lag preservation;
- negative-policy behavior;
- frequency-axis conversions;
- malformed lag, window, padding, and option rejection.

The focused release gate is:

```
pytest -q tests/test_spectral.py tests/test_velocity_spectrum.py tests/test_vacf.py
```

The complete package suite must also pass before packaging.

Reuse and non-reuse

Reused:

- VACFResult arrays, weights, atom indices, origin counts, and provenance;
- SciPy FFT and physical constants;
- the package's frozen dataclass/result-validation style.

Not reused:

- `_fft.linear_fft_length`, because it has a signal-correlation padding contract;
- `_fft.positive_lag_correlation_from_spectrum`, because it performs the inverse numerical direction;
- MSD FFT algebra;
- plotting normalization.

Direct Welch estimator (VS2)

Public function

```
def compute_velocity_spectrum(
    collection: AtomisticFrameCollection,
    *,
    species: SpeciesSelection = None,
    atom_indices: ArrayLike | None = None,
    weights: WeightInput = "uniform",
    drift_mode: DriftMode | None = None,
    drift_species: SpeciesSelection = None,
    drift_atom_indices: ArrayLike | None = None,
    segment_length: int | None = None,
    overlap: float | int = 0.5,
    window: str | tuple[str, float] = "hann",
    detrend: Literal["none", "constant"] = "none",
    zero_pad_to: int | None = None,
    compute_tensor: bool = True,
    per_atom: bool = False,
    per_atom_indices: ArrayLike | None = None,
    atom_block_size: int | None = None,
    memory_target_bytes: int = 256_000_000,
) -> VelocitySpectrumResult:
    ...
```

Motive

A transformed VACF uses the chosen lag estimator and remains sensitive to long-lag noise and truncation. The direct estimator provides an independent frequency-domain route with an explicit variance-resolution tradeoff. It is particularly useful for checking finite-record artifacts in VS1.

Borrowed algorithm

The numerical estimator is Welch's method [5]:

1. divide a uniformly sampled record into overlapping finite segments;
2. multiply every segment by a deterministic window;
3. form a modified periodogram by FFT;
4. average the segment periodograms.

`scipy.fft.rfft`, `scipy.fft.rfftfreq`, and `scipy.signal.get_window` supply the FFT and DFT-even window implementations [6, 7]. The `mdstats` implementation does not call `scipy.signal.welch` internally because it must aggregate atom-resolved self spectra, tensor cross-components, and memory-bounded atom blocks under one normalization. SciPy `welch` and `csd` are independent test oracles.

mdstats contribution and attribution boundary

The following are `mdstats` designs rather than claims about Welch's original paper:

- use of AtomisticFrameCollection and the shared `_velocity_common` contract;
- physical center-of-mass or center-of-geometry drift removal before segments;
- optional segmentwise constant detrending as a separate operation;
- multiplication by $\sqrt{q_i}$ followed by self-only atom aggregation;
- no products between distinct atoms;
- atom-blocked execution through `make_atom_spectrum_plan()`;
- Cartesian Hermitian tensor output and request-ordered per-atom results;
- frozen result validation and complete reproducibility metadata.

Segment and overlap contract

If `segment_length` is `None`, the resolved length is

$$N_{\text{seg}} = \min(256, N_{\text{sample}}),$$

matching the conventional SciPy default scale while never exceeding the record. An explicit segment length must satisfy

$$2 \leq N_{\text{seg}} \leq N_{\text{sample}}.$$

A floating overlap is a fraction in $[0, 1)$ and resolves through

$$N_{\text{overlap}} = \lfloor \text{overlap} N_{\text{seg}} \rfloor.$$

An integer overlap is an exact sample count. The segment advance is

$$A = N_{\text{seg}} - N_{\text{overlap}} > 0,$$

and starts are

$$0, A, 2A, \dots$$

while a complete segment remains. A trailing incomplete segment is not padded or averaged and its discarded sample count is recorded.

Window and detrend contract

The segment window is constructed with

`scipy.signal.get_window(window, segment_length, fftbins=True)`

and must have finite positive squared energy

$$U = \sum_{n=0}^{N_{\text{seg}}-1} w_n^2 > 0.$$

The default is a periodic Hann window and 50% overlap. Window name, parameter, DFT-even convention, sum, and squared sum are stored in metadata.

Physical drift removal uses `_velocity_common` and is applied to the full trajectory reference frame. It is distinct from `detrend="constant"`, which subtracts each atom/component segment mean before windowing. Constant detrending can suppress genuine low-frequency transport and therefore remains disabled by default.

Weighted self-only periodograms

For segment s , measured atom i , and Cartesian component α ,

$$X_{si\alpha}(f_m) = \text{RFFT} [w_n \sqrt{q_i} v_{si\alpha}(n)]_m.$$

The raw Cartesian component density is

$$P_{\alpha\alpha}(f_m) = \frac{1}{N_s f_s U} \sum_s \sum_i |X_{si\alpha}(f_m)|^2,$$

where $f_s = 1/\Delta t$ and N_s is the number of complete segments. The tensor uses

$$P_{\alpha\beta}(f_m) = \frac{1}{N_s f_s U} \sum_s \sum_i X_{si\alpha}(f_m)^* X_{sj\beta}(f_m).$$

Only equal-atom products occur. In particular, the implementation never forms $X_{si\alpha}^* X_{sj\beta}$ for $i \neq j$; this function is a self velocity spectrum, not a collective current spectrum.

The one-sided scale doubles all positive interior bins and leaves DC and, for even FFT length, Nyquist unchanged. normalization is stored as "raw": the weighted atom sum is not divided by weight_sum. Uniform and explicit weights therefore give units of $\text{\AA}^2/\text{ps}$, while mass weighting gives $\text{amu} \text{\AA}^2/\text{ps}$.

FFT length and frequency grid

The FFT length is

```
next_fast_len(max(segment_length, zero_pad_to or segment_length))
```

so it is never shorter than a segment. Zero padding refines the frequency grid but does not improve the physical resolution set by segment duration. The canonical grid is

$$f_m = \frac{m}{N_{\text{FFT}} \Delta t}$$

in cycles/ps, numerically THz.

Atom blocking

`make_atom_spectrum_plan()` selects a measured-atom block. For each segment and block the implementation:

1. gathers canonical atom velocities;
2. subtracts the framewise drift if requested;
3. optionally subtracts the segment mean;
4. multiplies by $\sqrt{q_i}$ and the segment window;
5. computes one real FFT over the segment axis;
6. accumulates component, tensor, and requested per-atom self periodograms.

Estimator-wide arrays remain independent of block size. Different block sizes may change the final floating-point reduction order by roundoff but must agree to numerical tolerance.

Algorithm

1. Validate booleans, detrending, trajectory semantics, velocities, and uniform time sampling.
2. Resolve measured atoms, weights, physical drift, and per-atom outputs through `_velocity_common.py`.
3. Resolve segment length, overlap, advance, starts, periodic segment window, and FFT length.
4. Create the N3.1 memory plan.
5. Loop deterministically over segment starts and atom blocks.
6. form weighted windowed block FFTs;
7. accumulate component powers and optional tensor cross-components;
8. retain requested per-atom powers without introducing cross-atom products;
9. divide by segment count, sampling frequency, and window energy;
10. apply one-sided density scaling;
11. construct all frequency axes and validate `VelocitySpectrumResult`.

Metadata contract

The result records at least:

- measured, drift-reference, and per-atom selections;
- weight mode and units;
- velocity source and trajectory source files;
- segment length, overlap input, overlap count, advance, starts, count, and discarded tail;
- window specification, DFT-even convention, sum, and squared energy;
- detrending mode and physical drift mode;

- sampling frequency, density denominator, FFT length, and padding request;
- atom-block size, estimated block workspace, and memory target;
- explicit `self_terms_only=True` and `cross_atom_products_included=False`.

Input constraints and failures

VS2 rejects:

- non-trajectory collections, missing velocities, or nonuniform time grids;
- segment lengths outside the available record;
- negative, full-segment, nonfinite, or otherwise invalid overlaps;
- unsupported or zero-energy window specifications;
- detrend modes other than none and constant;
- invalid atom selections, weights, drift references, or per-atom subsets;
- FFT padding shorter than one sample or noninteger padding;
- invalid atom-block or memory-target arguments.

A measured subset used as its own drift reference emits the existing `CollectiveMotionVACFWarning`. Finite-difference velocities emit the existing `FiniteDifferenceVelocityWarning` because high-frequency amplitudes may be attenuated.

VS2 tests

The focused VS2 suite verifies:

- scalar PSD agreement with `scipy.signal.welch`;
- tensor cross-spectrum agreement with `scipy.signal.csd`;
- tensor Hermiticity and component/trace identities;
- full-record boxcar equivalence to the biased VS1 VACF transform;
- exact self-only per-atom summation without cross-atom contamination;
- atom-block invariance to floating-point tolerance;
- mass-weighted units and weight sums;
- constant detrending removal of a DC signal;
- selection and per-atom request ordering;
- tuple windows, overlap resolution, one-atom memory fallback, and invalid-input rejection.

The focused release gate is

```
pytest -q \
  tests/test_velocity_spectrum.py \
  tests/test_spectral.py \
  tests/test_velocity_common.py \
  tests/test_vacf.py \
  tests/test_vdos.py \
  tests/test_plot_velocity_spectrum.py
```

Deferred work

This stage does not implement:

- VDOS normalization;
- spectrum plotting;
- Green-Kubo integration;
- peak fitting or linewidth extraction;
- infrared or Raman intensities.

Those remain later units in the VACF/dynamics roadmap.

H0 signature propagation and deep immutability

`compute_vacf_spectrum()` preserves the source `VACFResult.signature`. `compute_velocity_spectrum()` preserves the signature constructed by shared velocity preparation. A spectral transform changes representation, not the physical trajectory, atom selection, drift-reference population, or velocity source.

All frequency axes, spectra, tensor/per-atom arrays, and nested metadata are owned/read-only or recursively frozen. The signature itself is deeply immutable and must agree with the measured atoms and full Cartesian source subspace. Strict shared validators reject boolean substitutes for integer controls and require actual booleans for switches migrated through velocity preparation.

These requirements are normative through `_dynamics_common_spec.md`; they do not alter VS1/VS2 spectral normalization or existing numerical values.

References

- [1] N. Wiener, “Generalized Harmonic Analysis,” *Acta Mathematica* **55**, 117-258 (1930). DOI: 10.1007/BF02546511.
- [2] A. Khintchine, “Korrelationstheorie der stationären stochastischen Prozesse,” *Mathematische Annalen* **109**, 604-615 (1934). DOI: 10.1007/BF01449156.
- [3] A. Rahman, “Correlations in the Motion of Atoms in Liquid Argon,” *Physical Review* **136**, A405-A411 (1964). DOI: 10.1103/PhysRev.136.A405.
- [4] F. J. Harris, “On the Use of Windows for Harmonic Analysis with the Discrete Fourier Transform,” *Proceedings of the IEEE* **66**, 51-83 (1978). DOI: 10.1109/PROC.1978.10837.
- [5] P. D. Welch, “The Use of Fast Fourier Transform for the Estimation of Power Spectra: A Method Based on Time Averaging over Short, Modified Periodograms,” *IEEE Transactions on Audio and Electroacoustics* **15**, 70-73 (1967). DOI: 10.1109/TAU.1967.1161901.
- [6] P. Virtanen, R. Gommers, T. E. Oliphant, et al., “SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python,” *Nature Methods* **17**, 261-272 (2020). DOI: 10.1038/s41592-019-0686-2.
- [7] SciPy Developers, `scipy.signal.welch`, `scipy.signal.csd`, and `scipy.signal.get_window`, SciPy Reference Guide, accessed 2026-07-15.